

Machine Learning II

TUHH, Winter Term 2025/26

Practical Exercises P2 (Ay, Datar, Röder, October 30, 2025)

Exercise P2.1 (*Regression with a Markov Kernel*, sum of 15 points)

In this exercise, you will implement a neural network that performs a regression task where the conditional distribution of the target variable y given the input x is modeled as a Gaussian with input-dependent mean and variance. Thus, the regression model corresponds to learning a Markov kernel

$$x \mapsto p_\theta(\cdot | x) = \mathcal{N}(y | \mu_\theta(x), \sigma_\theta(x)^2),$$

Consider the target function $f : [-2, 2] \rightarrow \mathbb{R}$ defined by

$$f(x) = 2 \sin(3x) + 0.5x^3 - x^2 + 1 + 0.5 \sin(20x),$$

with input-dependent noise $\eta(x) \sim \mathcal{N}(0, \sigma^*(x)^2)$ where $\sigma^*(x) = 0.1 + 0.2|x|$.

(a) **(8 points)** Implement a neural network with two output nodes

- Architecture: input dimension 1, $N = 8$ hidden neurons with ReLU activation, two outputs: $\mu_\theta(x)$ and $\log \sigma_\theta(x)$ (log standard deviation)
- Generate $m = 50$ training samples by sampling $x \in [-2, 2]$ uniformly and for each x_i , generate $n = 10$ points y_j from the distribution $\mathcal{N}(f(x_i), \sigma^*(x_i)^2)$
- Use Gaussian negative log-likelihood (NLL) loss:

$$\mathcal{L}_z(\theta) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \left[\log \sigma_\theta(x_i) + \frac{(y_j - \mu_\theta(x_i))^2}{2\sigma_\theta(x_i)^2} \right]$$

- Implement gradient descent with a small learning rate and compute gradients manually using backpropagation (no automatic differentiation) to minimize $\mathcal{L}_z$

Important: You must derive the gradients of the NLL loss with respect to both $\mu_\theta(x)$ and $\log \sigma_\theta(x)$ and then use the chain rule to implement back propagation. Furthermore, consider different weight and bias initializations.

(b) **(5 points)** Visualize the learned density estimation:

- Plot the predicted mean $\mu_\theta(x)$ and 95% confidence interval $[\mu_\theta(x) - 2\sigma_\theta(x), \mu_\theta(x) + 2\sigma_\theta(x)]$
- Overlay the true mean function $f(x)$ and training data points
- Plot the training loss (NLL) evolution on an appropriate scale
- Generate and visualize samples from the learned distribution $p_\theta(y|x)$

(c) **(2 points)** Compute and visualize the pointwise KL divergence:

$$D_{KL}(p^*(y|x) || p_\theta(y|x)) = \log \frac{\sigma_\theta(x)}{\sigma^*(x)} + \frac{\sigma^*(x)^2 + (\mu_\theta(x) - f(x))^2}{2\sigma_\theta(x)^2} - \frac{1}{2}$$

Plot D_{KL} as a function of x and discuss where the conditional distributions are well approximated.

Exercise P2.2 (*Binary Classification with Neural Networks*, sum of 15 points) In this exercise, you will implement a neural network for binary classification using the Iris dataset. The goal is to classify whether an iris flower is of the species *Iris versicolor* or not, based on four features: sepal length, sepal width, petal length, and petal width.

(a) **(8 points)** Implement a binary classification neural network:

- Architecture: input dimension 4, $N = 64$ hidden neurons with ReLU activation, output dimension 1 with sigmoid activation
- Load the Iris dataset from `iris.csv` (provided in StudIP)
- Create binary labels: $y_i = 1$ if the flower is *versicolor*, $y_i = 0$ otherwise
- Use binary cross-entropy (BCE) loss:

$$\mathcal{L}_z(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log p_\theta(x_i) + (1 - y_i) \log(1 - p_\theta(x_i))]$$

where $p_\theta(x_i)$ is the output of the neural network representing the predicted probability of the sample being the species *Iris versicolor*

- Implement gradient descent, manually computing gradients using backpropagation (no automatic differentiation). Select an appropriate learning rate and number of epochs for training.

(b) **(5 points)** Visualize the classification results:

- Plot the decision boundary in 2D using petal length and petal width (fixing sepal features to their mean values)
- Scatter plot the training data points colored by their true class
- Show the decision boundary as a contour line where $p_\theta(x) = 0.5$
- Plot the training loss (BCE)

(c) **(2 points)** Analyze the learned classifier:

- Compute and report the final training accuracy: $\frac{1}{m} \sum_{i=1}^m \mathbb{1}[(\hat{y}_i = y_i)]$ where $\hat{y}_i = \mathbb{1}[p_\theta(x_i) \geq 0.5]$